Phishing URL Detector

An Explainable Rule-Based Tool for Identifying Malicious Links

Project Report

BharatCares × IBM SkillsBuild
Cyber Security Program

Submitted by: Chhitiz Srivastava
October 2026

1. Abstract

Phishing remains one of the most widespread cyber attacks. Attackers send links that imitate banks, payment apps, e-commerce sites and government services to steal credentials, OTPs and money. This project presents **Phishing URL Detector**, a Python command-line tool that analyses the structure of a URL and assigns it a risk score from 0 to 100. Fourteen independent checks cover transport security, IP-based hosts, deceptive characters, subdomain abuse, risky top-level domains, URL shorteners, suspicious keywords, brand impersonation (typosquatting using edit distance and leetspeak normalisation), obfuscation and domain randomness. The tool is explainable: every point added to the score is displayed with its reason. It uses only the Python standard library, runs offline and never visits the URL, which makes it safe for testing and teaching.

2. Introduction

Phishing is a form of social engineering in which a victim is tricked into opening a fake website or sharing sensitive data. Indian users are frequently targeted through SMS and WhatsApp messages claiming a bank account will be blocked, a KYC update is pending, a refund is due or a prize has been won. Because a mobile browser hides most of the address, many people cannot judge whether a link is genuine.

Security teams rely on layered defence: user awareness, blocklists, machine learning and manual analysis. A simple, transparent URL scanner is a valuable learning tool because it shows precisely which properties of a link are suspicious.

2.1 Objectives

- Build a tool that classifies a URL as SAFE, SUSPICIOUS or PHISHING.
- Make every decision explainable to the user.
- Detect look-alike domains of popular Indian and global brands.
- Keep the tool lightweight, offline and free of third-party dependencies.

3. Problem Statement

Given a URL string, determine without visiting it whether it is likely to be a phishing link, and report the evidence supporting the decision.

4. Requirements

4.1 Functional

- Accept a URL from the command line, a text file or interactive input.
- Normalise URLs that lack a scheme.
- Compute a risk score and verdict.
- Print the list of triggered indicators with their weights.

4.2 Non-functional

- **Portability:** Python 3.8+ on Windows, Linux and macOS.
- **Safety:** only text analysis; no network requests to the target.
- **Performance:** a result in milliseconds per URL.
- **Maintainability:** each check is a small, independent block.

4.3 Software and hardware

Item	Specification
Language	Python 3.8 or newer
Libraries	argparse, re, math, ipaddress, urllib (standard library)
Optional	reportlab (only to regenerate this PDF)
Hardware	Any computer with 512 MB RAM

5. System Design

The tool follows a simple four-stage pipeline.

Stage	Description
1. Input	URL read from argument, file or prompt; scheme added if missing.
2. Parse	urllib splits scheme, host, port, path and query. The host is split into subdomains, registered domain and T
3. Analyse	Fourteen checks run independently; each returns points and a reason.
4. Score & report	Points are summed (max 100), mapped to a verdict and printed with reasons.

5.1 Scoring thresholds

Score	Verdict	Meaning
0 - 24	SAFE	No significant red flags
25 - 49	SUSPICIOUS	Some risky traits; verify before trusting
50 - 100	PHISHING	Strong indicators of a malicious link

6. Detection Methodology

#	Check	Weight	Rationale
1	Not HTTPS	10	Traffic is unencrypted; modern legitimate sites use HTTPS.
2	IP address host	25	Real services use domain names.
3	'@' in URL	25	Everything before '@' is ignored by browsers, hiding the true host.
4	URL length	5-10	Long URLs push the real domain out of view.
5	Subdomain depth	5-15	Fake domain buried among many labels.
6	Hyphens / digits	5-10	Typical of throw-away registrations.
7	High-risk TLD	15	Free or cheap TLDs heavily abused (.tk, .xyz, .top).
8	URL shortener	25	Destination is hidden.
9	Suspicious keywords	5 each, max 20	login, verify, kyc, otp, refund, etc.
10	Brand impersonation	20-35	Typosquat, embedded brand, or brand as subdomain.
11	Obfuscation	5-20	Punycode, percent-encoding, double slash.
12	Odd port	10	Public sites seldom use ports other than 80/443.
13	High entropy domain	10	Randomly generated names (Shannon entropy > 3.5).
14	Many query params	5	Redirect and tracking abuse.

6.1 Brand look-alike detection

The registered domain label is converted to lower case and common character substitutions are reversed (0→o, 1→l, 3→e, 4→a, 5→s, 7→t, @→a, $→s). The result is compared with a list of frequently impersonated brands such as PayPal, SBI, HDFC, Paytm, Amazon and IBM. A match after normalisation, or a Levenshtein edit distance of 1, is treated as typosquatting. Brands embedded in a

longer domain or placed in a subdomain of an unrelated domain are also flagged.

6.2 Entropy

Shannon entropy $H = -\Sigma\ p(c)\ \log_2\ p(c)$ is calculated over the characters of the domain label. Algorithmically generated domains tend to have higher entropy than human-chosen names.

7. Implementation

The program is a single file, `phishing_detector.py`, of about 250 lines. Key functions are listed below.

Function	Purpose
normalise(url)	Adds http:// when no scheme is present.
registered_parts(host)	Returns subdomains, domain label and TLD.
levenshtein(a, b)	Edit distance for typosquatting detection.
shannon_entropy(text)	Randomness measure of a domain.
analyse(url)	Runs all checks and returns score, verdict and findings.
report(result)	Formats the output for the terminal.
main()	Command-line interface (single URL, file or interactive).

7.1 Core scoring logic (excerpt)

```
score = min(sum(p for p, _ in findings), 100)
verdict = ("SAFE" if score < 25
           else "SUSPICIOUS" if score < 50
           else "PHISHING")
return {"url": raw, "score": score,
        "verdict": verdict, "findings": findings}
```

8. Testing and Results

The tool was run on ten URLs: three legitimate, one shortener and six crafted phishing patterns. All were classified as expected.

URL	Score	Verdict
https://www.google.com/search?q=cyber+security	0	SAFE
https://www.ibm.com/skills	0	SAFE
https://skillsbuild.org/students	0	SAFE
http://secure-login.paypa1.com/verify?account=1	65	PHISHING
http://192.168.10.5:8080/signin/update-password	68	PHISHING
http://sbi-kyc-update.xyz/otp/confirm	85	PHISHING
https://bit.ly/3xYzAbC	25	SUSPICIOUS
http://amazon.account-verify.login.secure-update.tk/signin	90	PHISHING
http://www.faceb00k.com/login	50	PHISHING
http://user@evil-site.top/bank/login	60	PHISHING

8.1 Sample explanation

```
[X] http://amazon.account-verify.login.secure-update.tk/signin
    Risk score : 90/100   Verdict : PHISHING
      +10  Connection is not HTTPS
      +15  Excessive subdomains (3)
```

```
+5   Hyphen in domain name
+15  High-risk TLD '.tk'
+20  Suspicious keywords: login, verify, update, secure...
+25  Brand 'amazon' used as subdomain of 'secure-update.tk'
```

Observation: the tool correctly flags typosquats (*paypa1*, *faceb00k*) and decoy-subdomain attacks while leaving genuine domains (google.com, ibm.com, skillsbuild.org) at a score of 0. A URL shortener alone is rated SUSPICIOUS because the destination cannot be verified.

9. Limitations

- Rule-based detection cannot learn new attack styles and may produce false positives on unusual but legitimate sites.
- The page content, domain age, SSL certificate details and reputation databases are not examined.
- The brand list is limited and must be updated manually.
- Weights were chosen from domain knowledge, not tuned on a large labelled dataset.

10. Future Scope

- Train a machine-learning model (e.g. Random Forest) on public phishing datasets using the same features.
- Add WHOIS domain-age and TLS certificate checks.
- Integrate PhishTank or Google Safe Browsing look-ups.
- Provide a Flask web interface and a browser extension.
- Expand the brand list with regional banks and government portals.

11. Learning Outcomes and User Safety Tips

Through this project I learnt how phishing URLs are constructed, how to parse and analyse URL components, how typosquatting is detected using string distance, and how to present security findings in an explainable way.

- Never click links in unexpected SMS, WhatsApp or e-mail messages; type the official address yourself.
- Banks never ask for OTP, PIN or password over a link or call.
- Check the real domain just before the first single slash, not the words at the start.
- Report suspicious links to cybercrime.gov.in or call 1930 in India.

12. Conclusion

The Phishing URL Detector demonstrates that a compact, transparent set of checks can identify many common phishing patterns and teach users what to look for. While not a replacement for enterprise-grade protection, it is a practical foundation that can be extended with machine learning and threat-intelligence feeds.

13. References

- Anti-Phishing Working Group (APWG) - Phishing Activity Trends Reports. apwg.org
- OWASP - Phishing and Social Engineering resources. owasp.org
- Indian Cyber Crime Coordination Centre (I4C) - cybercrime.gov.in
- Python documentation - urllib.parse, ipaddress. docs.python.org
- IBM SkillsBuild - Cybersecurity learning pathways. skillsbuild.org